

neon-hydro-colocated-water-cycle-python-myst

April 21, 2026

This tutorial explores hydrologic data products published by NEON across observational, instrumented, and remote sensing subsystems. It focuses on downloading and exploring various NEON hydrologic data products. It also dives deeper into NEON hydrology data, using co-located terrestrial-aquatic sites to examine the water cycle at multiple levels.

0.1 Learning Objectives

After completing this activity, you will be able to:

- Download and explore the contents of NEON hydrologic data products from the observational and instrumented subsystems.
- Navigate to tools and data sources for more derived data products such as geospatial data, bundled eddy-covariance data, or data from the airborne observation platform.
- Understand the similarities and linkages between different NEON data products.
- Join and plot hydrologic data sets from the instrumented subsystem across a terrestrial-aquatic gradient at NEON co-located sites.

Requirements: Python 3.9+

0.2 Set up: Install Packages

Packages only need to be installed once, you can skip this step after the first time:

- **os**: Module allowing interaction with user's operating system
- **pandas**: Module for working with data frames
- **neonutilities**: Basic functions for accessing NEON data
- **matplotlib**: Functions for plotting
- **geopy**: Compute distances between latitude/longitude coordinates
- **plotly**: Functions for producing interactive plots
- **statsmodels**: Functions for the estimation of statistical models

```
[ ]: !pip install os
      !pip install pandas
      !pip install neonutilities
      !pip install matplotlib
      !pip install geopy
      !pip install plotly
      !pip install statsmodels
```

0.3 Additional Resources

- [Tutorial for using neonUtilities from both R and Python environments](#)
- [GitHub repository for neonUtilities](#)
- [neonUtilities cheat sheet](#) - A quick reference guide for users.
- [HydroShare Collection - NEON Hydrologic Data Products: Site-Level Resources](#)
- [CUAHSI Cyberseminar Series: Introduction to NEON for Hydrology - An Overview of Hydrologic Data Products and Tools](#)

0.4 Set up: Load Packages

```
[ ]: import os
import pandas as pd
import neonutilities as nu
import matplotlib.pyplot as plt
import matplotlib.dates as mdates
from geopy.distance import geodesic
import plotly.graph_objects as go
import statsmodels.api as sm
import numpy as np
```

0.5 Set NEON Data Portal API Token

It is recommended that NEON data users have a NEON Data Portal API token set as an environment variable. See [this tutorial](#) for instructions on obtaining a NEON API token.

```
[ ]: os.environ.setdefault('NEON_PAT', "YOUR_API_TOKEN_HERE")
```

0.6 Download & Explore: Introduction

In this tutorial, we will focus on one pair of co-located NEON sites from Domain 07 - Appalachians & Cumberland Plateau:

- [Oak Ridge National Laboratory \(ORNL\) - Terrestrial](#)
- [Walker Branch \(WALK\) - Aquatic](#)

But, the workflow can be replicated for any pair of co-located sites across the observatory that contain all the data products used in this tutorial, which is defined by the following criteria:

- Aquatic site has published discharge data and has groundwater wells installed.
- Terrestrial site has published precipitation data.
 - *Note:* Precipitation data are available from two different data products depending on the collection method at a site. Check the following data products to ensure you are downloading the correct data product for a site:
 - * [Precipitation - weighing gauge \(DP1.00044.001\)](#)
 - * [Precipitation - tipping bucket \(DP1.00045.001\)](#)
- Terrestrial and aquatic site are within 10-km of each other.

The following site pairs meet that criteria:

NEON Domain	Aquatic Site	Terrestrial Site
D02	LEWI	BLAN
D02	POSE	SCBI
D03	FLNT	JERC
D06	KING	KONA
D07	WALK	ORNL
D08	TOMB	LENO
D08	BLWA	DELA
D08	MAYF	TALL
D12	BLDE	YELL
D13	COMO	NIWO
D16	MART	WREF
D17	BIGC	SOAP

In this tutorial, we will focus on one water year of data: Water Year 2024, defined as 2023-10-01 through 2024-09-30. And, for each data product used in this tutorial, we will download data published in RELEASE-2026. To learn more about the differences between released and provisional data, see the [Understanding Releases and Provisional Data](#) tutorial on the NEON website.

0.7 Download & Explore: Instrumented (IS) Data Products

For this exercise, we will download and explore one data product from the instrumented subsystem: [Precipitation - tipping bucket \(DP1.00045.001\)](#). This data product is published from NEON's terrestrial sites; thus, we will use the NEON site code 'ORNL'.

```
[ ]: # Download precipitation data for a single water year
ptp_py = nu.load_by_product(dpid="DP1.00045.001",
                             site="ORNL",
                             startdate="2023-10",
                             enddate="2024-09",
                             release="RELEASE-2026",
                             package="expanded",
                             check_size=False,
                             token=os.environ.get("NEON_PAT"))
```

Downloads from the NEON Utilities packages contain multiple files, including data tables, meta-data, and data product documentation. Let's explore each set of files in turn.

0.8 Files Associated with IS Downloads

The data we've downloaded comes as an object that is a dictionary of objects. Let's view the contents of the download package.

```
[ ]: # Get all file names in the download package
ptp_py.keys()
```

In this tutorial, we downloaded the **expanded** download package. What are the files contained in this download package and why are they useful?

- **TIPPRE_1min** and **TIPPRE_30min**: Includes the primary data tables of the Precipitation - tipping bucket data product. We will dive deeper into data tables in the next section.
- **sensor_positions_00045**: Reports the geolocation of each sensor included in the download.
- **science_review_flags_00045**: Lists each science review flag (SRF) date range, flag value, and justification applied to the data included in this download.
- **issueLog_00045**: Reports issues that may impact data quality, or changes to a data product that affects one or more sites.
- **variables_00045**: This file contains all the variables found in the data table(s) included in this download. This includes full definitions, units, and other important information.
- **readme_00130**: The readme file provides important information relevant to the data product and the specific instance of downloading the data.
- **citation_00045_RELEASE-2026**: Formatted citation and DOI for the data included in this download.

0.9 Explore IS Data Tables

The expanded download package for DP1.00045.001 contains two data tables that report precipitation time series data, each reporting data at a different temporal resolution:

- **TIPPRE_1min**: Tipping bucket precipitation reported at a 1-min resolution
- **TIPPRE_30min**: Tipping bucket precipitation averaged to a 30-min resolution

Below, we will explore the first few rows of **TIPPRE_30min**. Add to the code below to also view other tables included in the expanded download package.

```
[ ]: # Print the first 5 records in the time series data
ptp_py['TIPPRE_30min'].head()
```

0.10 Explore IS Variables

The **variables_00045** file provides insight into the structure of each data table and associated variables included in a download package. View the variables file and familiarize yourself with the different fields, data types, and units used in this data product.

```
[ ]: # View variables file to understand data table structure
ptp_py['variables_00045']
```

0.11 Download & Explore: Observational (OS) Data Products

Let us now explore a hydrologic data product from the observational subsystem. For this exercise, we move to the surface component of the hydrologic cycle. We will download and explore hydrologic data derived from surface water grab samples: [Stable isotopes in surface water \(DP1.20206.001\)](#). This data product is published from NEON's aquatic sites; thus, we will use the NEON site code 'WALK'.

```
[ ]: # Download stable isotopes data for a single water year
asi_py = nu.load_by_product(dpid="DP1.20206.001",
                           site="WALK",
                           startdate="2023-10",
                           enddate="2024-09",
```

```
release="RELEASE-2026",
package="expanded",
check_size=False,
token=os.environ.get("NEON_PAT"))
```

Let's do the same exploration of the download as we did for the instrumented data product and see what is similar and different.

0.12 Files Associated with OS Downloads

```
[ ]: # Get all file names in the download package
asi_py.keys()
```

When we view the content of the observational data product download, we notice similarities and differences relative to the instrumented data product. For example, both data products include `citation`, `variables`, `issuelog`, and `readme` files. What do we notice that is different?

- The observational data product does not contain `science_review_flags` or `sensor_positions` files. Those files are specific to instrumented data products.
- Files specific to observational data products are included:
 - `categoricalCodes_20206`: Some variables in the data tables are published as strings and constrained to a standardized list of values (LOV). This file shows all the LOV options for variables published in this data product.
 - `validation_20206`: If any fields require validation prior to publication, those validation rules are reported in this table.
- There are many more data tables published in this observational data product. Let's explore that in the next section.

0.13 Explore Data OS Tables

For this sample-based observational data product, there are many more tables published than the previous instrumented data product we explored. That is because data are collected and published at each point along the lifetime of a sample, from collection to analysis. Let's break down the table structure for this stable isotopes data product.

- `asi_fieldSuperParent`: Field data associated with the 'superparent' water sample, which is a 4-L grab samples that, once subsampled, results in multiple observational data products, including this stable isotopes data product.
- `asi_fieldData`: Field data associated with the stable isotopes subsample.
- `asi_externalLabH2OIsotopes`: Results of hydrogen-2 and oxygen-18 stable isotope ratio analysis in filtered surface water samples.
- `asi_externalLabSummaryData`: Accuracy and precision data for the instrument used in the analysis of H2O stable isotopes.
- `asi_POMExternalLabDataPerSample`: Results of carbon-13 and nitrogen-15 stable isotope ratio analysis in particulate organic matter (POM) filtered out of surface water samples.
- `asi_externalLabPOMSummaryData`: Accuracy and precision data for the instrument used in the analysis of POM stable isotopes.

For this exercise, let's just explore the first few rows of `asi_externalLabH2OIsotopes`. Add to the code below to also view other tables included in the expanded download package.

```
[ ]: # Print the first 5 records in H2O stable isotope lab data
asi_py['asi_externalLabH2OIsotopes'].head()
```

0.14 Explore OS Variables

```
[ ]: # View variables file to understand data table structure
asi_py['variables_20206']
```

0.15 Download & Explore: Higher-Level Hydrologic Data Products

NEON data products are processed at progressive levels. The precipitation and stable isotopes data products are Level 1 data products, which is the lowest level of data processing required for a NEON data product. Higher level hydrologic data products exist that include additional processing in the form of spatial and/or temporal interpolation, or the incorporation of algorithms or scientific theory to derive higher-order quantities.

[More information on NEON data processing levels.](#)

For this exercise, we will introduce three high-level hydrologic data products and show how to download them.

0.16 Higher-Level Hydrologic Data Products: Stream morphology maps

The [Stream morphology maps \(DP4.00131.001\)](#) data product is a Level 4 aquatic data product published at all NEON stream sites. The data product includes many data tables with post-processed survey data and links to geospatial data and site maps stored in the cloud. Let's download the data product and fetch the geospatial data from the cloud.

```
[ ]: # Download stream morphology data for the lifetime of a site
# URL to download geospatial data from the cloud is stored in geo_surveySummary
geo_py = nu.load_by_product(dpid="DP4.00131.001",
                           site="WALK",
                           release="RELEASE-2026",
                           package="basic",
                           check_size=False,
                           token=os.environ.get("NEON_PAT"))
```

```
[ ]: # Get the URL for the most recent geomorphology survey
# Copy and paste the URL to your browser to retrieve the data package
print(max(geo_py['geo_surveySummary']['dataFilePath'][
    geo_py['geo_surveySummary']['surveyBoutTypeID']=="geomorphology"
]))
```

0.17 Higher-Level Hydrologic Data Products: Net Surface-Atmosphere Exchange (Eddy Covariance)

The net surface-atmosphere exchange data products are available for all terrestrial sites and are bundled together in a single Level 4 data product: [Bundled data products - eddy covariance](#)

(DP4.00200.001). The data packages do not contain comma separated tabular data. Rather, the data and metadata are stored as HDF5 files.

To download and view bundled eddy covariance data, you cannot use the standard `load_by_product()` function. You must use a combination of other functions available in the NEON Utilities package.

```
[ ]: # Download eddy covariance data for a single water year
nu.zips_by_product(dpid="DP4.00200.001",
                  site="ORNL",
                  startdate="2023-10",
                  enddate="2024-09",
                  release="RELEASE-2026",
                  package="basic",
                  check_size=False,
                  token=os.environ.get("NEON_PAT"))

[ ]: # Stack the data download, parse to data frames and read into environment
# defaults to stacking only L4 products
sae_py = nu.stack_eddy(filepath = "filesToStack00200")

[ ]: # The data is stored by site name - print the header of the 'ORNL' table
sae_py["ORNL"].head()
```

0.18 Higher-Level Hydrologic Data Products: Canopy Water Indices - Mosaic

The [Canopy water indices - mosaic \(DP3.30019.001\)](#) is a Level 3 (spatially-interpolated) data product published from the Airborne Observation Platform (AOP) subsystem.

Remote sensing data products are large, but NEON has developed many tools to aid users in downloading and interpreting AOP data. We will not download AOP data in this exercise. Rather, follow the links below for guides on downloading AOP data in R, Python, and Google Earth Engine (GEE).

- [Download and Explore NEON Data](#) - Directly links to 'Download remote sensing data: by-FileAOP() and byTileAOP()' section.
- [Intro to AOP Data in Google Earth Engine \(GEE\) Tutorial Series](#)
- [Understanding AOP Data Releases and Best Practices for AOP Data Management](#)

0.19 Merge & Visualize: The Water Cycle at Co-Located Sites

In this exercise, we will merge together three hydrologic data products, each from a different section of the water cycle at NEON co-located sites:

- [Precipitation - tipping bucket \(DP1.00045.001\)](#)
- [Elevation of groundwater \(DP1.20100.001\)](#)
- [Continuous discharge \(DP4.00130.001\)](#)

We will download each data product, identify how each product can be related, then merge the three data streams into a single data frame.

0.20 Download Data Products

This time, we will download the basic download packages.

```
[ ]: # Download precipitation data for a single water year
ptp_py = nu.load_by_product(dpid="DP1.00045.001",
                             site="ORNL", # Terrestrial data product
                             startdate="2023-10",
                             enddate="2024-09",
                             release="RELEASE-2026",
                             package='basic',
                             check_size=False,
                             token=os.environ.get("NEON_PAT"))
```

```
[ ]: # Download groundwater elevation data for a single water year
egw_py = nu.load_by_product(dpid="DP1.20100.001",
                             site="WALK", # Aquatic data product
                             startdate="2023-10",
                             enddate="2024-09",
                             release="RELEASE-2026",
                             package='basic',
                             check_size=False,
                             token=os.environ.get("NEON_PAT"))
```

```
[ ]: # Download discharge data for a single water year
csd_py = nu.load_by_product(dpid="DP4.00130.001",
                             site="WALK", # Aquatic data product
                             startdate="2023-10",
                             enddate="2024-09",
                             release="RELEASE-2026",
                             package='basic',
                             check_size=False,
                             token=os.environ.get("NEON_PAT"))
```

0.21 Identify Relational Data & Merge

Due to the standardized spatial and temporal designs of NEON data products, these three instrumented data products can be related and merged in a relatively easy fashion.

Temporal Relationships

- All three data products have the same temporal structure. They all have the columns `startDateTime` and `endDateTime` in the data tables, and the columns are all formatted the same in published data: YYYY-MM-DD HH:MM:SS (UTC).
- All three data products are published at a similar temporal frequency. The precipitation and groundwater elevation products are each published at a 30-min resolution and the discharge product is published at a 15-min resolution.
- Therefore, merging the three tables by one of the datetime columns will ensure the data will be temporally related.

Spatial Relationships

- For this pair of co-located sites, the precipitation and discharge data products are published at one location, but the groundwater elevation data product is published at multiple locations.
- For this exercise, we will use the groundwater well location that is closest to the discharge locations. This information can easily be parsed by comparing sensor location coordinates in the `sensor_positions` file included in each data download.

```
[ ]: # In this download, there are 3 well locations that publish elevation
# There is only 1 location for discharge
# Use `geopy` to identify which well location is closest to discharge
from geopy.distance import geodesic
egw_coords = egw_py['sensor_positions_20100'].
    ↪drop_duplicates(subset=['locationReferenceLatitude'],
    ↪['locationReferenceLongitude'])
csd_coords = csd_py['sensor_positions_00130']
dist = egw_coords.apply(lambda row: geodesic((row['locationReferenceLatitude'],
    ↪row['locationReferenceLongitude']),
    (csd_coords.
    ↪iloc[0]['locationReferenceLatitude'], csd_coords.
    ↪iloc[0]['locationReferenceLongitude'])).meters, axis=1)
```

```
[ ]: # Which well is closest to the discharge location (horizontal position - HOR)?
close_loc = egw_coords.loc[dist.idxmin(), 'HOR.VER']
```

```
[ ]: # Let's use only the data from the closest well (subset by HOR)
egw_df = egw_py['EOG_30_min'][egw_py['EOG_30_min']['horizontalPosition'] ==
    ↪close_loc[:3]] # First 3 digits = HOR
```

```
[ ]: # Merge 3 data streams into a single data frame
# Keep the relevant data needed to plot timeseries and examine relationships
ptp_df = ptp_py['TIPPRE_30min'][['endDateTime', 'precipBulk', 'finalQF']]
egw_df = egw_df[['endDateTime', 'groundwaterElevMean', 'gWatElevFinalQF']]
csd_df = csd_py['csd_15_min'][['endDateTime', 'dischargeContinuous',
    ↪'dischargeFinalQF']]
wc_df = pd.merge(ptp_df, egw_df, on='endDateTime', how='outer')
wc_df = pd.merge(wc_df, csd_df, on='endDateTime', how='outer')
wc_df = wc_df.sort_values('endDateTime')
```

0.22 Plot & Download Merged Interactive Timeseries

Now, let's plot the three data streams in a single plotting field. We will use the `plotly` package to give us the ability to interact with the plot. Check your current working directory for the HTML file containing the plot.

```
[ ]: # Create figure
wc_df_ts = wc_df.dropna(subset=['precipBulk'])
fig = go.Figure()
```

```

# Format axes and layout
fig.update_layout(
    margin=dict(l=90, r=110, t=40, b=60),
    xaxis=dict(
        domain=[0, 0.9],
        tickmode='auto',
        nticks=14,
        automargin=True,
        title="Date",
        tickfont=dict(size=16)
    ),
    yaxis=dict(
        title="Discharge (L s-1)",
        tickfont=dict(size=16),
        showgrid=False,
        zeroline=False
    ),
    yaxis2=dict(
        title="Groundwater Elevation (m)",
        tickfont=dict(size=16, color='#CC79A7'),
        showgrid=False,
        zeroline=False,
        overlaying='y',
        side='right'
    ),
    yaxis3=dict(
        title="Precipitation (mm)",
        tickfont=dict(size=16, color='#0072B2'),
        showgrid=False,
        zeroline=False,
        overlaying='y',
        side='right',
        anchor='free',
        position=0.98
    ),
    legend=dict(
        orientation="h",
        y=-0.15,
        font=dict(size=14)
    ),
    updatemenus=[
        dict(
            type='buttons',
            showactive=False,
            buttons=[
                dict(

```

```

        label='Scale Discharge\n- Linear -',
        method='relayout',
        args=[{
            'yaxis.type': 'linear',
            'yaxis.title': "Discharge (L s-1)"
        }]
    ),
    dict(
        label='Scale Discharge\n- Log -',
        method='relayout',
        args=[{
            'yaxis.type': 'log',
            'yaxis.title': "Discharge (L s-1) - log"
        }]
    )
]
)
]
)

# Add traces
fig.add_trace(go.Scatter(
    x=wc_df_ts['endTime'],
    y=wc_df_ts['dischargeContinuous'],
    mode='lines',
    name='Discharge',
    line=dict(color='black')
))
fig.add_trace(go.Scatter(
    x=wc_df_ts['endTime'],
    y=wc_df_ts['groundwaterElevMean'],
    mode='lines',
    name='GW Elevation',
    line=dict(color='#CC79A7'),
    yaxis='y2'
))
fig.add_trace(go.Scatter(
    x=wc_df_ts['endTime'],
    y=wc_df_ts['precipBulk'],
    mode='lines',
    name='Precipitation',
    line=dict(color='#0072B2'),
    yaxis='y3'
))

fig.write_html("NEON.D07.P.H.Q.WY2024.html")

```

0.23 Further Exploration: Cumulative Precipitation & Discharge

```
[ ]: # Generate smooth curves of cumulative precipitation and discharge
wc_df_subset = wc_df[~wc_df['precipBulk'].isna()].copy()
wc_df_subset['cumulativeP'] = wc_df_subset['precipBulk'].cumsum()
wc_df_subset['cumulativeQ'] = wc_df_subset['dischargeContinuous'].cumsum()
x = wc_df_subset['endDateTime']
x_numeric = x.astype(np.int64) # nanoseconds since epoch
lowess = sm.nonparametric.lowess
smoothP = lowess(wc_df_subset['cumulativeP'], x_numeric, frac=0.3)
smoothQ = lowess(wc_df_subset['cumulativeQ'] / 150, x_numeric, frac=0.3)

[ ]: # Plot cumulative precipitation & discharge together using matplotlib with 2
    ↪ y-axes
fig, ax1 = plt.subplots(figsize=(10, 6))
ax1.plot(x, smoothP[:,1], color='#0072B2', label='Cumulative Precipitation (mm)')
    ↪ LOESS')
ax1.set_xlabel('Date', fontsize=12)
ax1.set_ylabel('Cumulative Precipitation (mm)', color='#0072B2', fontsize=12)
ax1.tick_params(axis='y', labelcolor='#0072B2')
ax2 = ax1.twinx()
ax2.plot(x, smoothQ[:,1], color='black', label='Cumulative Discharge (L s-1)')
    ↪ LOESS')
ax2.set_ylabel('Cumulative Discharge (L s-1)', color='black', fontsize=12)
ax2.tick_params(axis='y', labelcolor='black')
fig.tight_layout()
plt.show()
```

0.24 Further Exploration: Correlation of Groundwater Elevation & Discharge

```
[ ]: # Plot scatterplots of one variable to another to assess correlation
# Create a continuous color scale by date to add the time-of-year dimension
dates = pd.to_datetime(wc_df['endDateTime'])
date_nums = mdates.date2num(dates)
fig, ax = plt.subplots(figsize=(10, 6))
scatter = ax.scatter(
    wc_df['groundwaterElevMean'],
    wc_df['dischargeContinuous'],
    c=date_nums,
    cmap='cool',
    alpha=0.6
)
ax.set_xlabel('Groundwater Elevation (m)', fontsize=12)
ax.set_ylabel('Discharge (L s-1)', fontsize=12)
cbar = plt.colorbar(scatter, ax=ax)
cbar.set_label('Date', fontsize=12)
tick_locs = cbar.get_ticks()
```

```
cbar.ax.set_yticklabels([mdates.num2date(t).strftime('%Y-%m-%d') for t in
    ↪tick_locs])
plt.tight_layout()
plt.show()
```